

Reinforcement Learning

Part I

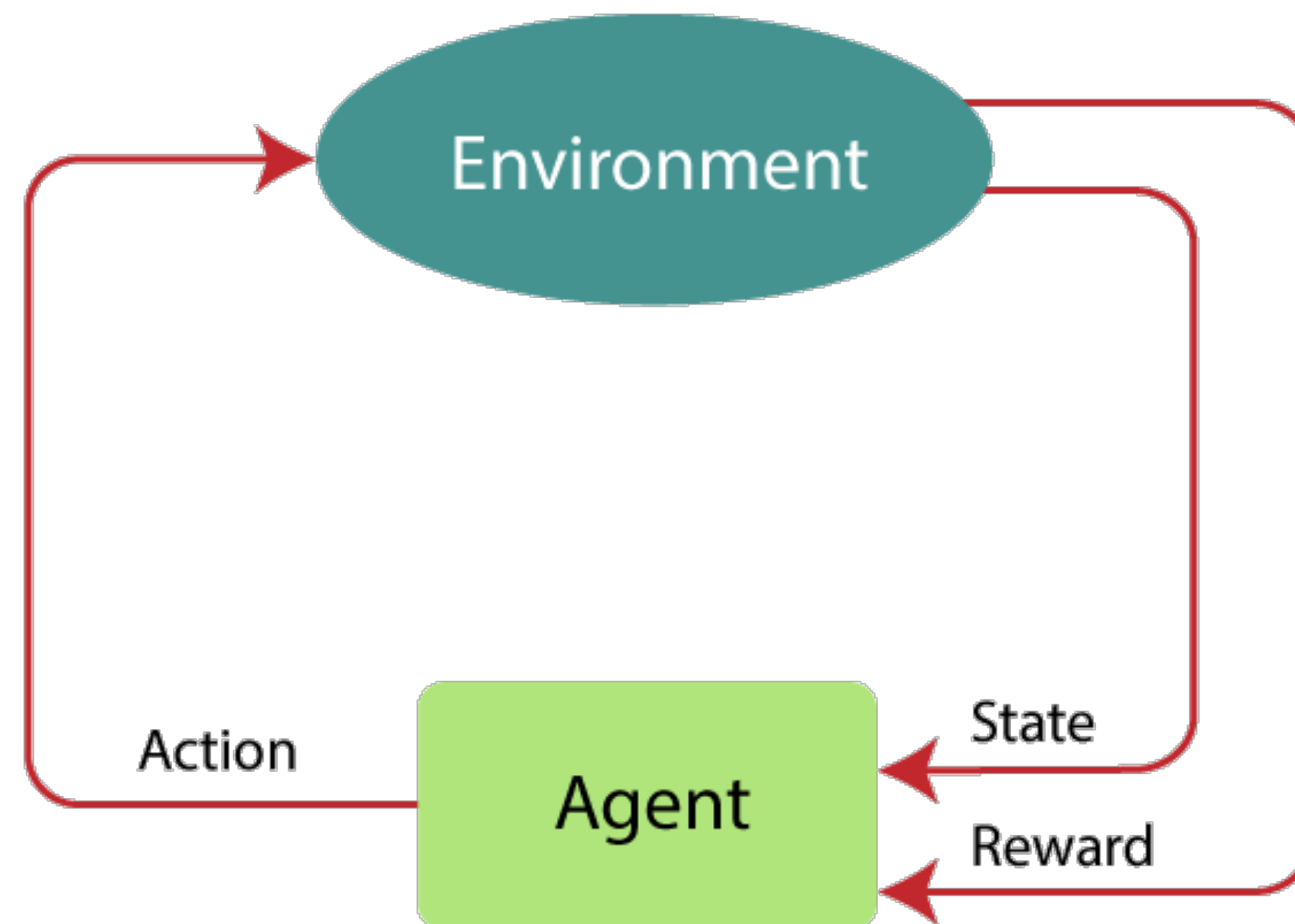
Prepared by: Joseph Bakarji

Example: OpenAI's Multi-Agent Hide and Seek



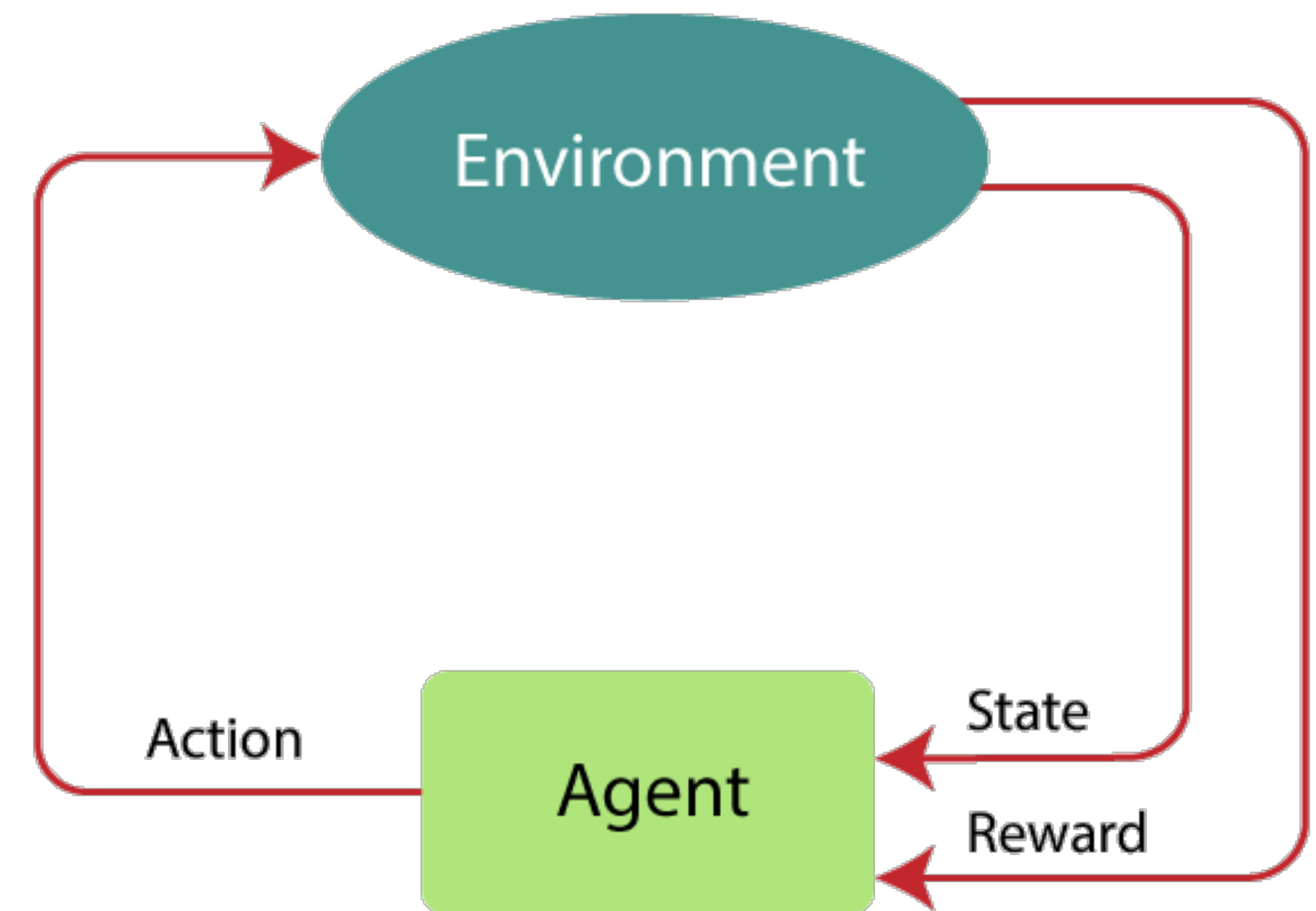
What are the features of an intelligent agent?

- **Actions:** make decisions and act from a set of possible
- **Perceptions:** observe the world and one's own state

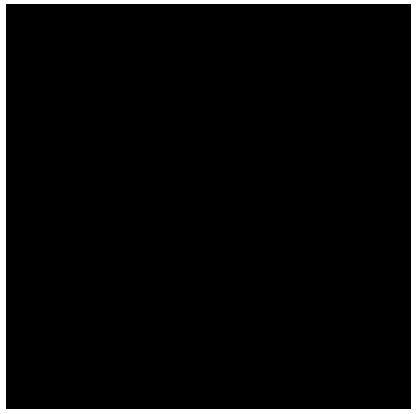


What are the features of an intelligent agent?

- Plan for long-term consequences: decisions today will affect the state tomorrow and later, etc.
- Decisions will determine how much information you collect from the environment
- No or little supervision: collect more data interactively



Example

-5			 +1	
				 +5
		 +2		
				+20 

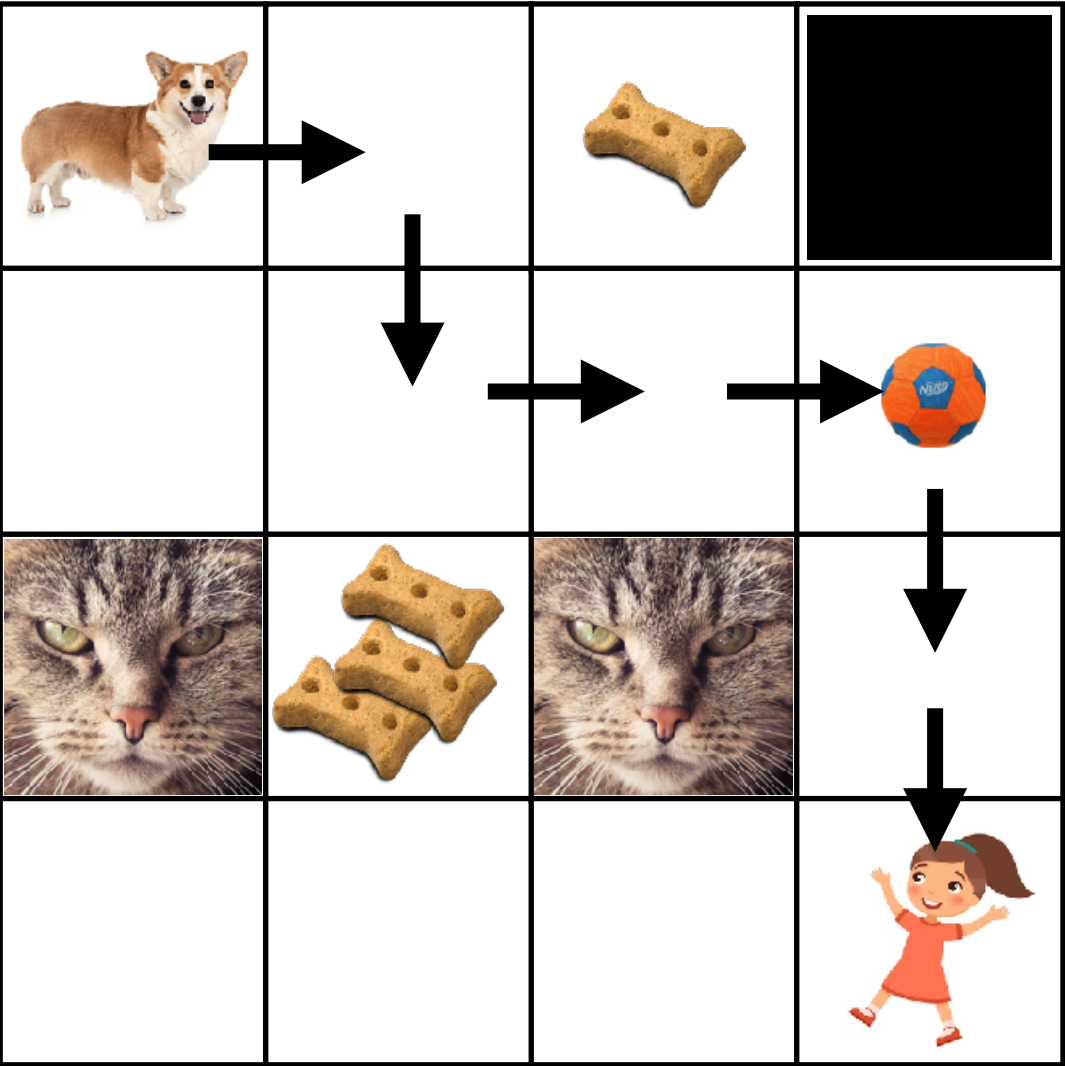
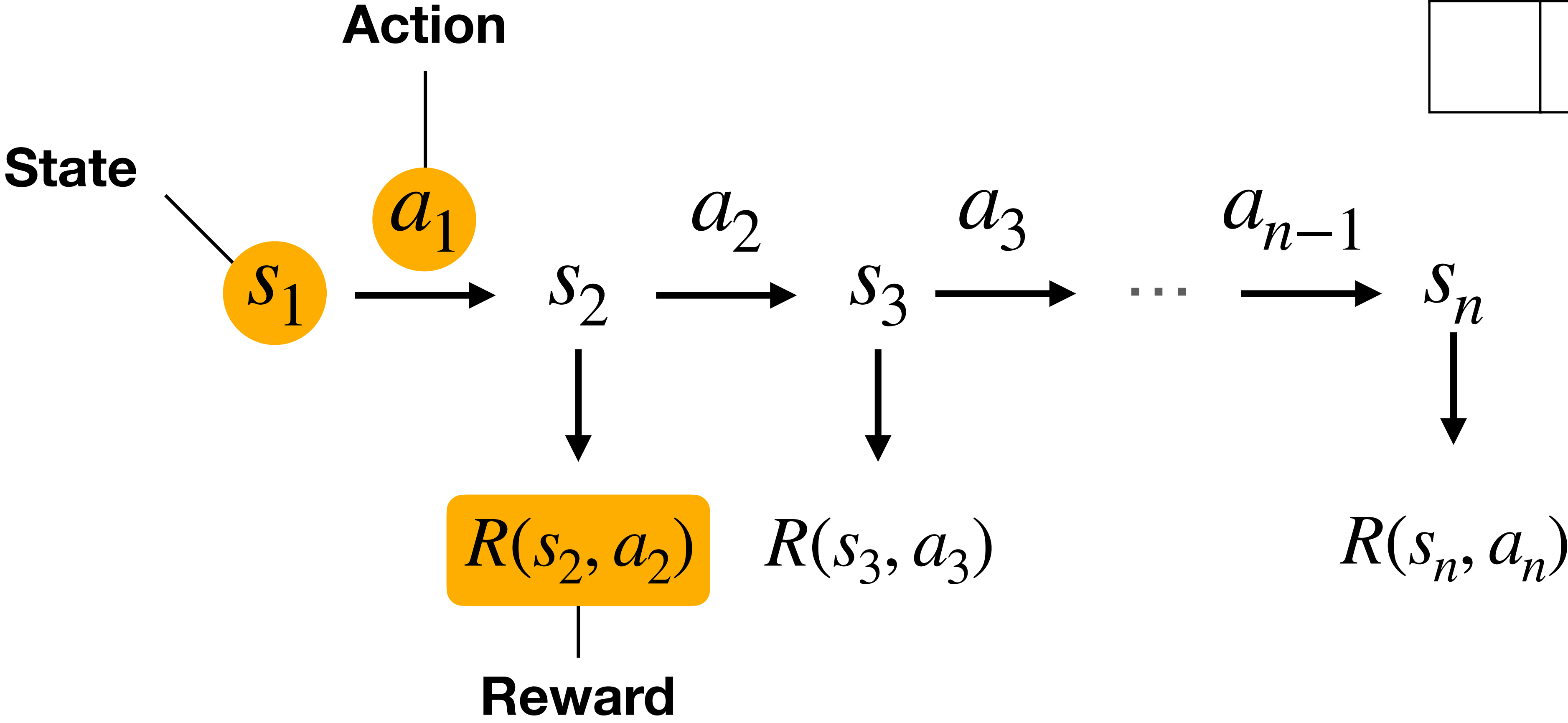
States: $s \in \{(1,1), (1,2), \dots, (4,4)\}$

Actions: $a \in \{ \rightarrow, \leftarrow, \uparrow, \downarrow \}$

Rewards: $R(s_i) \in [-5, 20]$

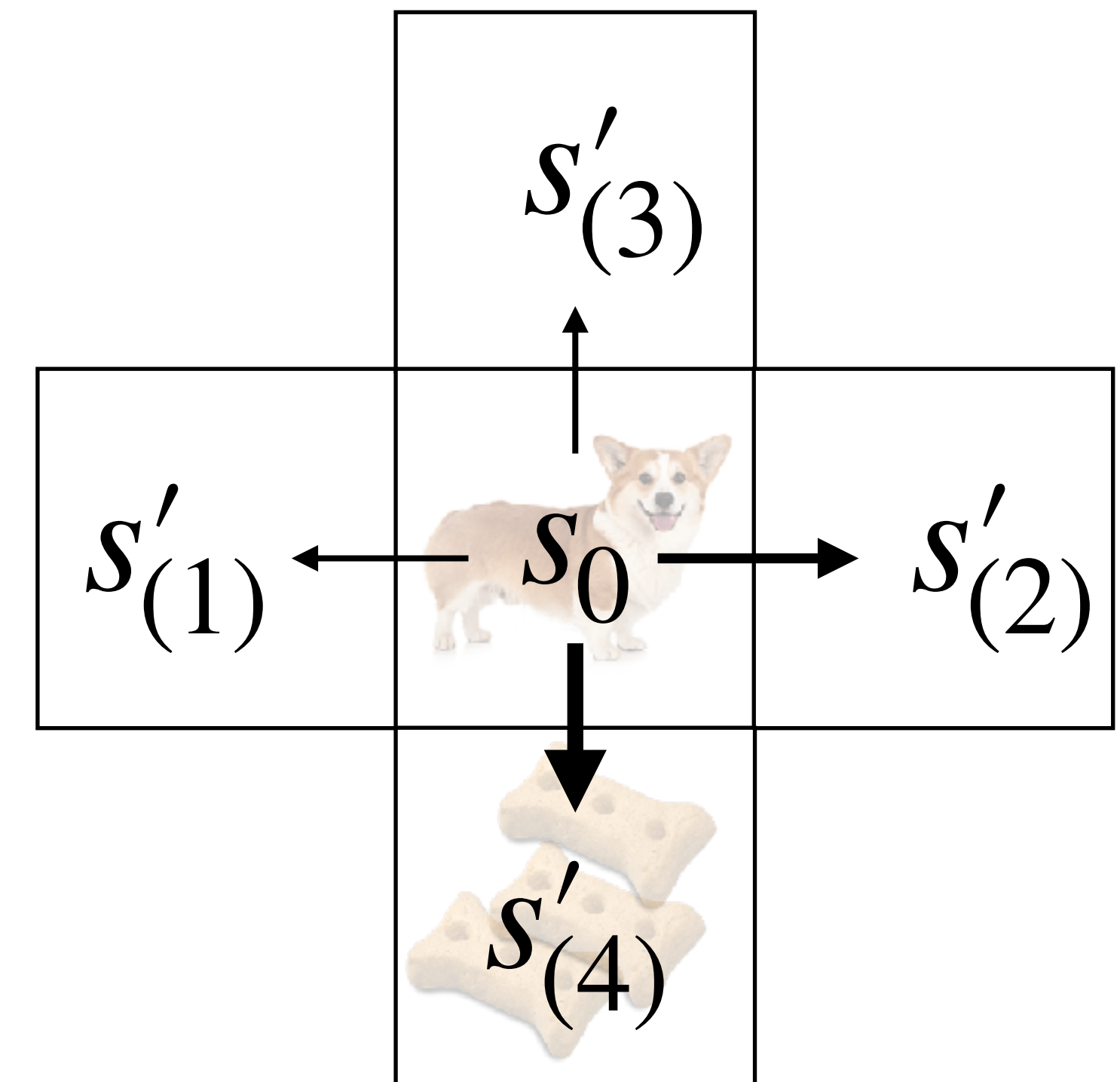
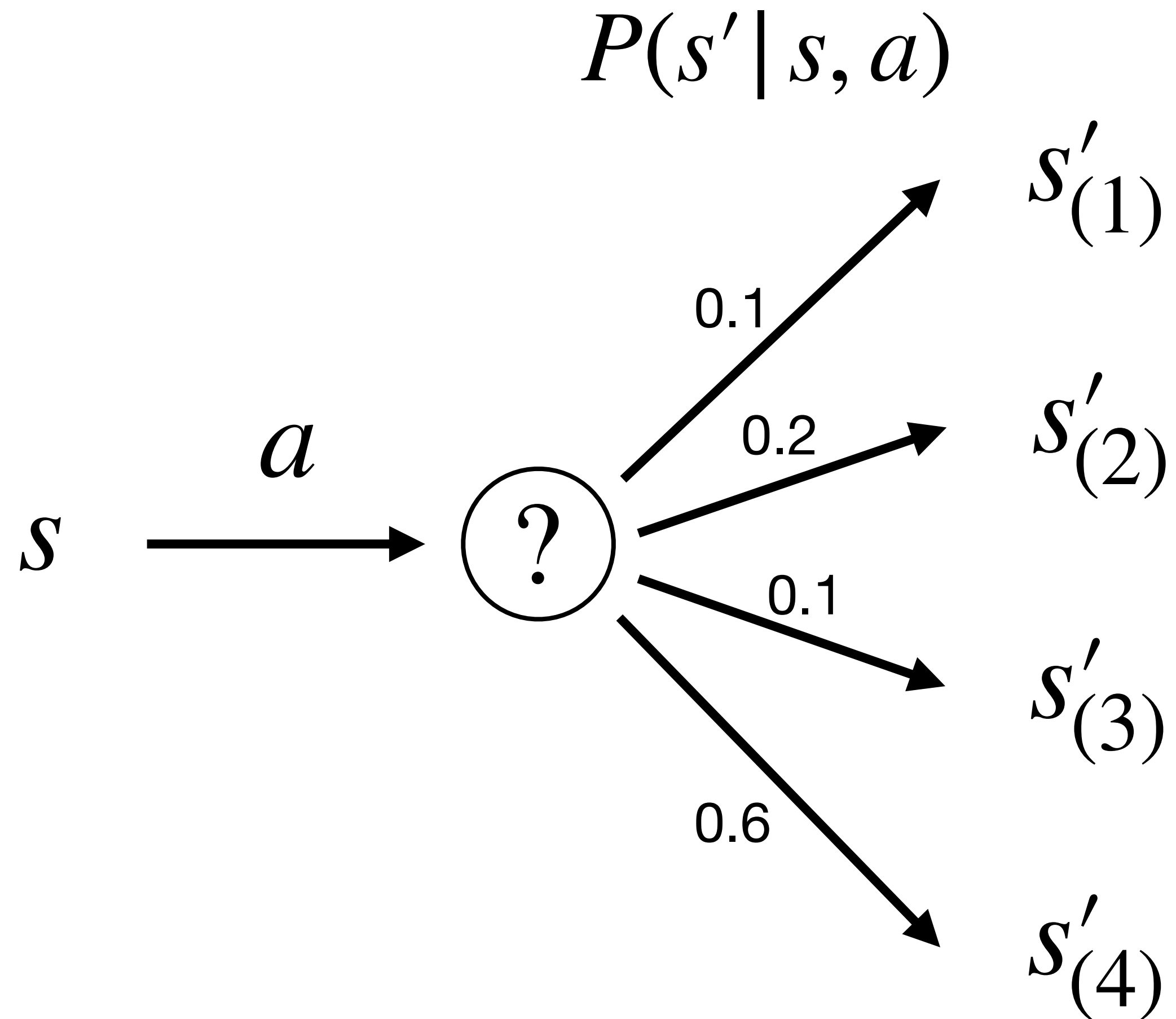
Transition probability: $P(s' | s, a) \in [0, 1]$

Definitions



Markov Decision Process (MDP)

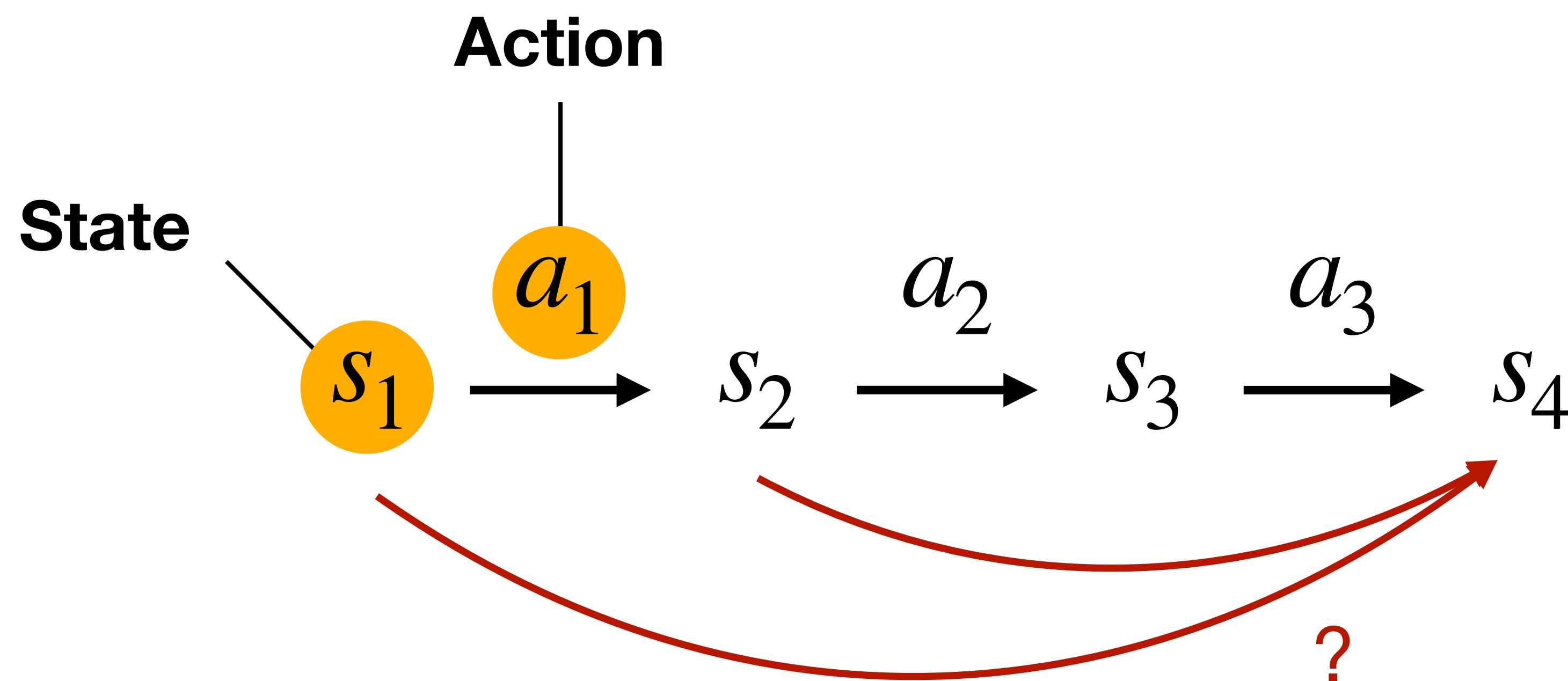
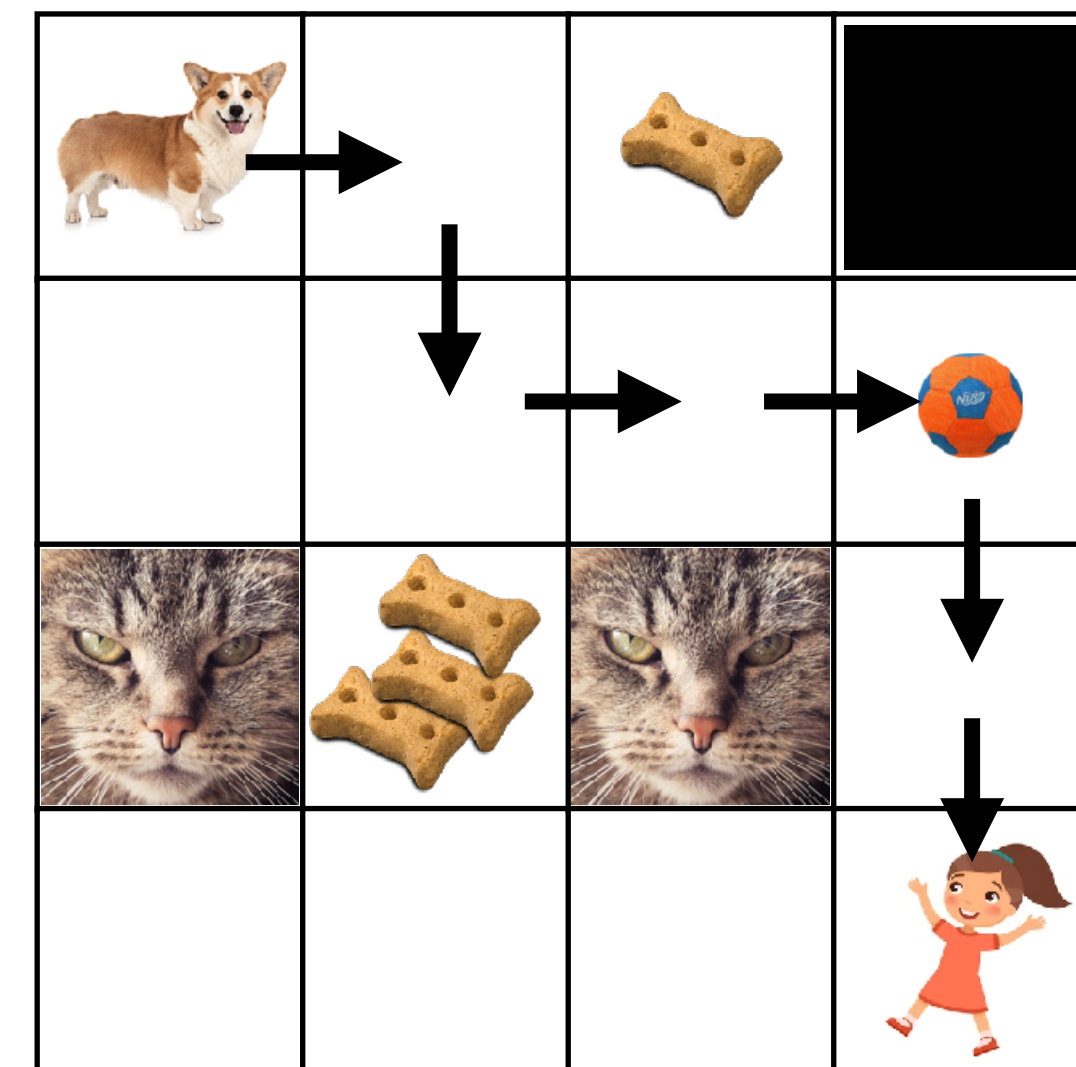
Assumes a probabilistic (uncertain) world



Markov Decision Process (MDP)

The **Markov assumption**:

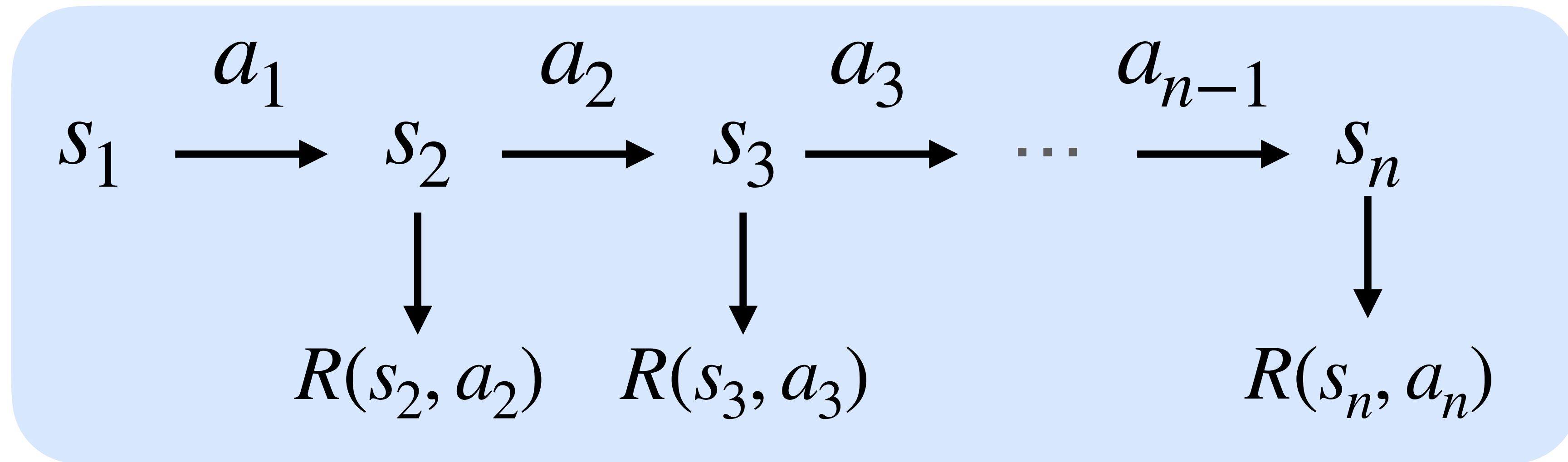
Transition probability only depends on previous state



$$P(s_i | s_{i-1}, s_{i-2}, s_{i-3}, a_i, a_{i-1}, \dots) = P(s_i | s_{i-1}, a_i)$$

Goal: Maximize Total Rewards

Definition of *Total Rewards*



$$\text{Total Rewards} = R(s_1, a_1) + \gamma R(s_2, a_2) + \gamma^2 R(s_3, a_3) + \dots$$

Utility

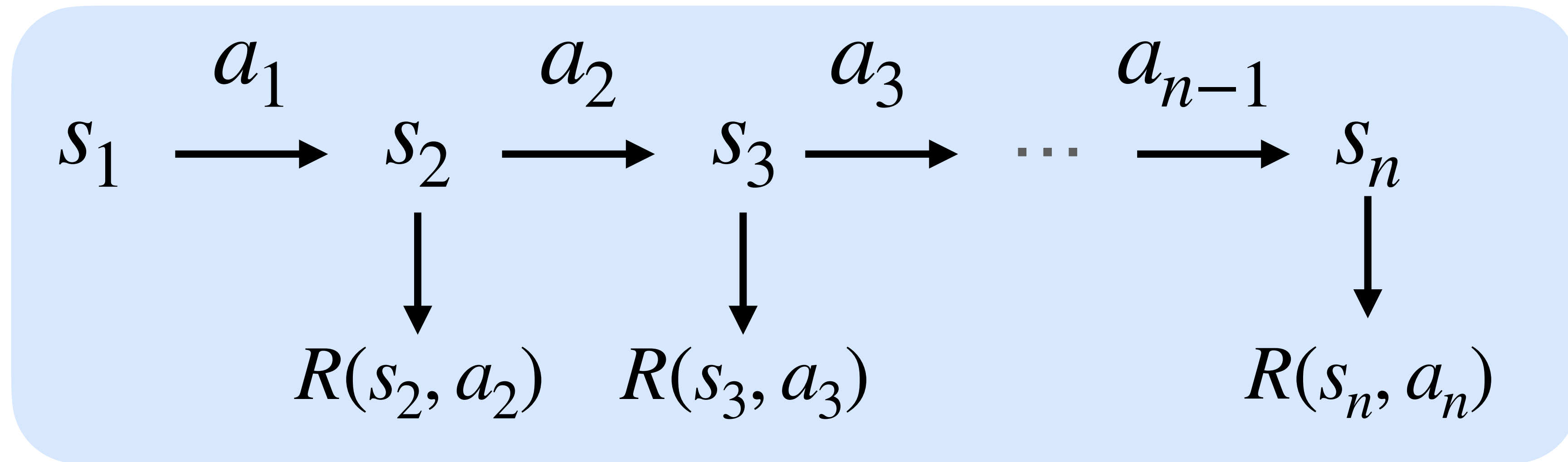
Discount Factor

$$\gamma < 1$$

Favors immediate rewards over future ones

Goal: Maximize Total Rewards

Definition of *Total Rewards*



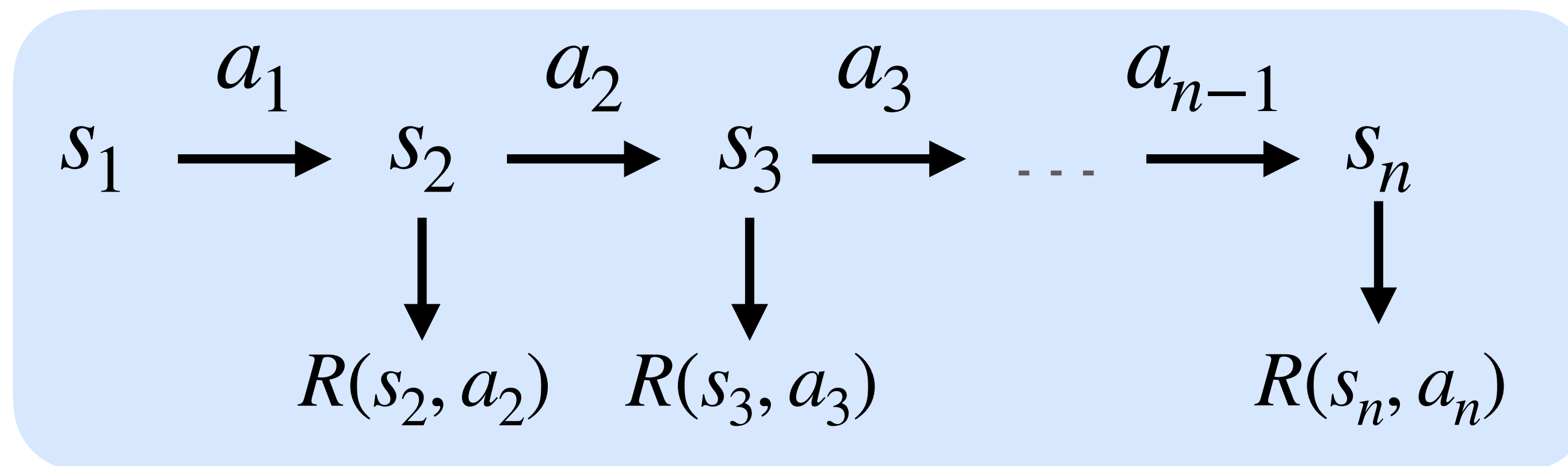
$$\text{Total Rewards} = R(s_1) + \underbrace{\gamma R(s_2)}_{\text{Discount Factor}} + \gamma^2 R(s_3) + \dots$$

Independence of actions

Favors immediate rewards over future ones

Goal: Maximize **Expectation** of Total Rewards

Given that decisions are probabilistic, we want to maximize the **expected** rewards



$$\text{Total Expected Rewards} = \mathbb{E} [R(s_1) + \gamma R(s_2) + \gamma^2 R(s_3) + \dots]$$

Expectation or average

Goal: Maximize **Expectation** of Total Rewards

Given that decisions are probabilistic, we want to maximize the **expected** rewards

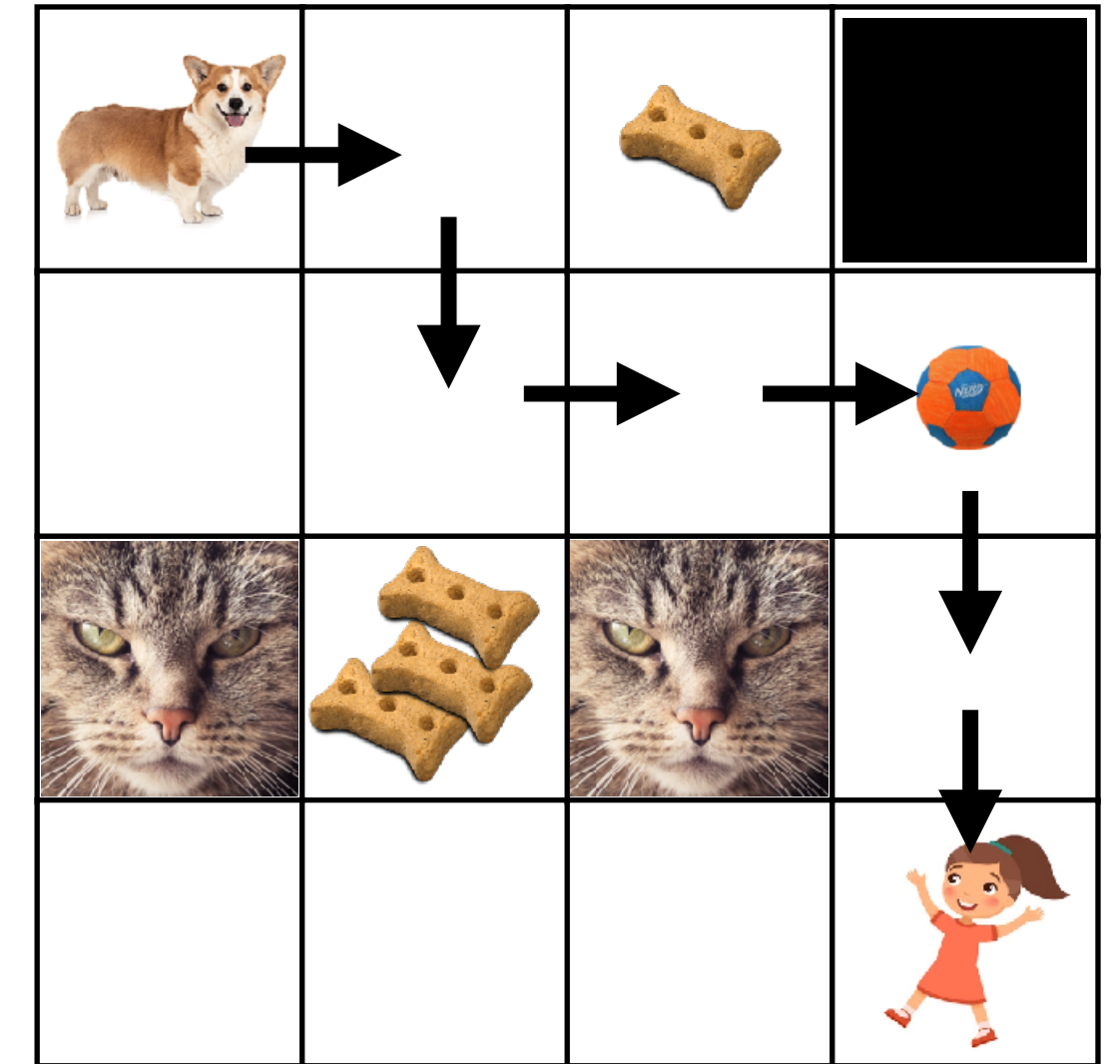
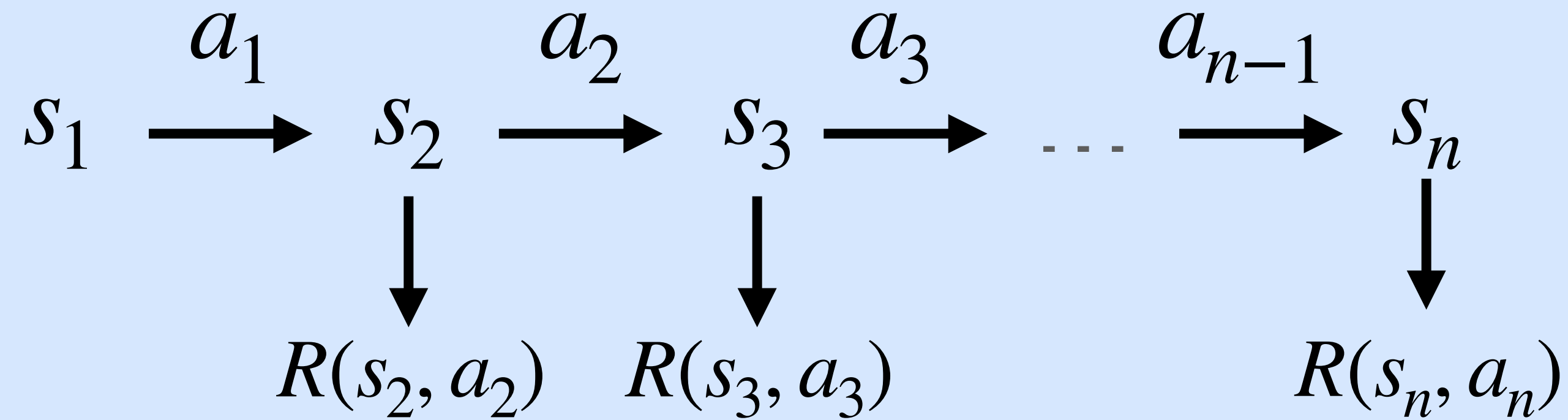
$$\text{Total Expected Rewards} = \mathbb{E} [R(s_1) + \gamma R(s_2) + \gamma^2 R(s_3) + \dots]$$

Expectation or Average

- Example: given a die with 6 faces, numbered 1 to 6. If you throw the die and get face 1, you make \$1, if you get 2, you make \$2, etc. What's your expected return?

$$\sum_i P(i)C(i) = \frac{1}{6}1 + \frac{1}{6}2 + \dots = \frac{21}{6}$$

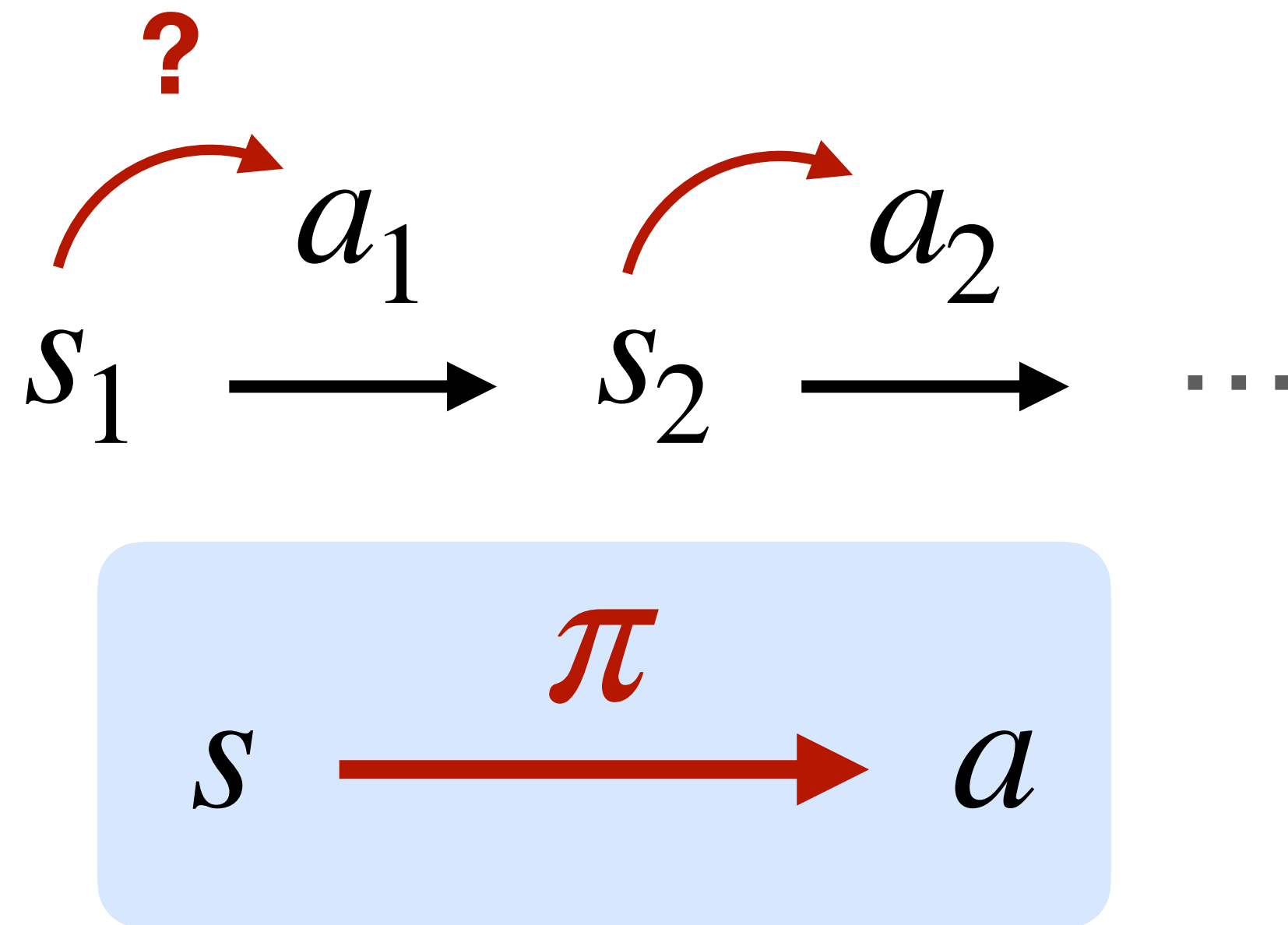
Markov Decision Process (MDP)



1. Set of **states**: S
2. Set of **actions**: A
3. State **transition probability**: $P(s' | s, a) \equiv P_{sa}(s')$, for $s, s' \in S$ and $a \in A$
4. **Reward** function: $R(s, a)$ or $R(s)$ that maps states (and actions) to a real number $\mathbb{R}$
5. **Discount** factor: $\gamma < 1$

How should the agent make decisions?

The agent needs a **policy** for taking action



We are executing some policy π if
whenever we're at s , we take action a according to

$$a = \pi(s)$$

How **good** is a policy π ?

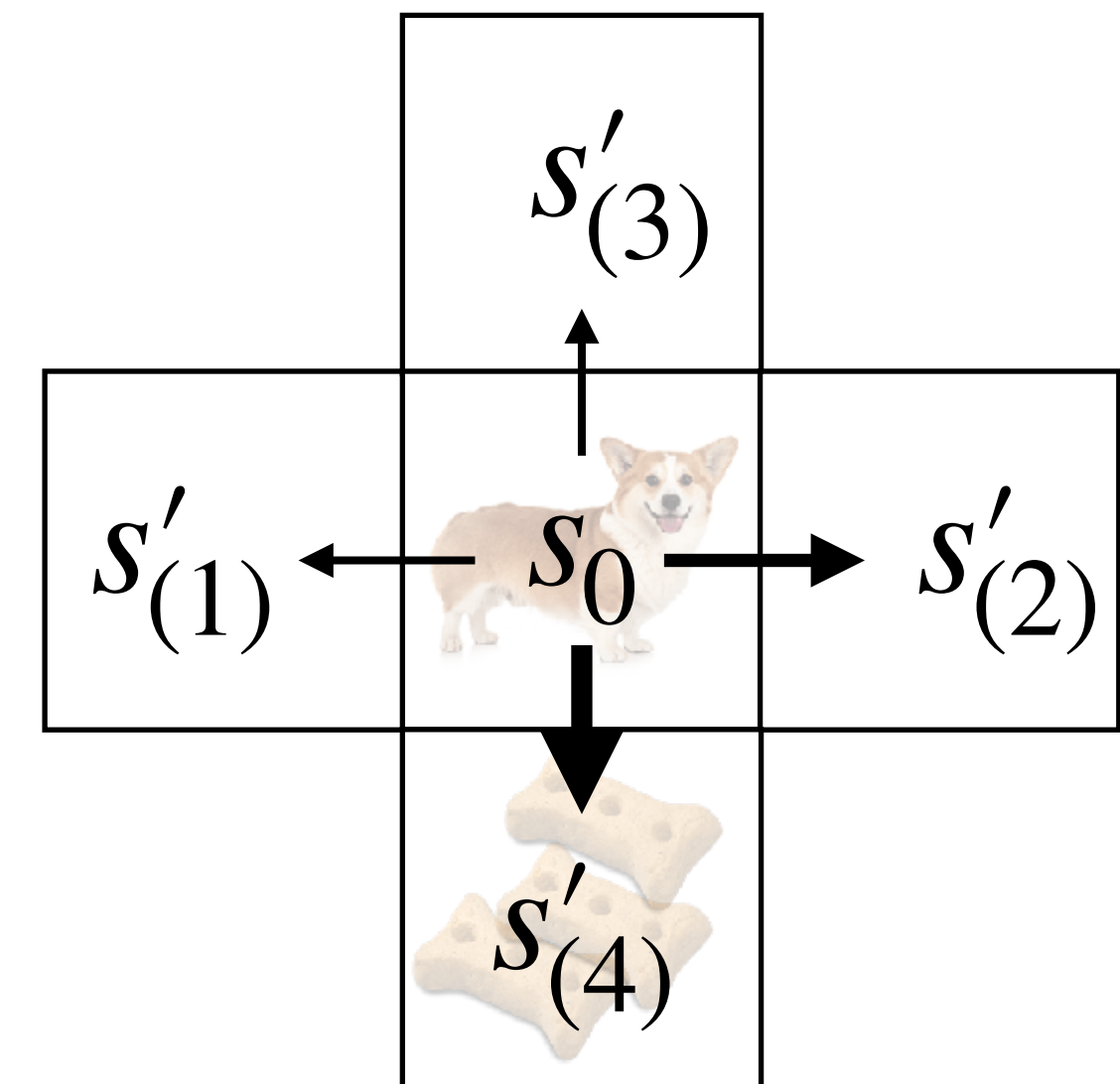
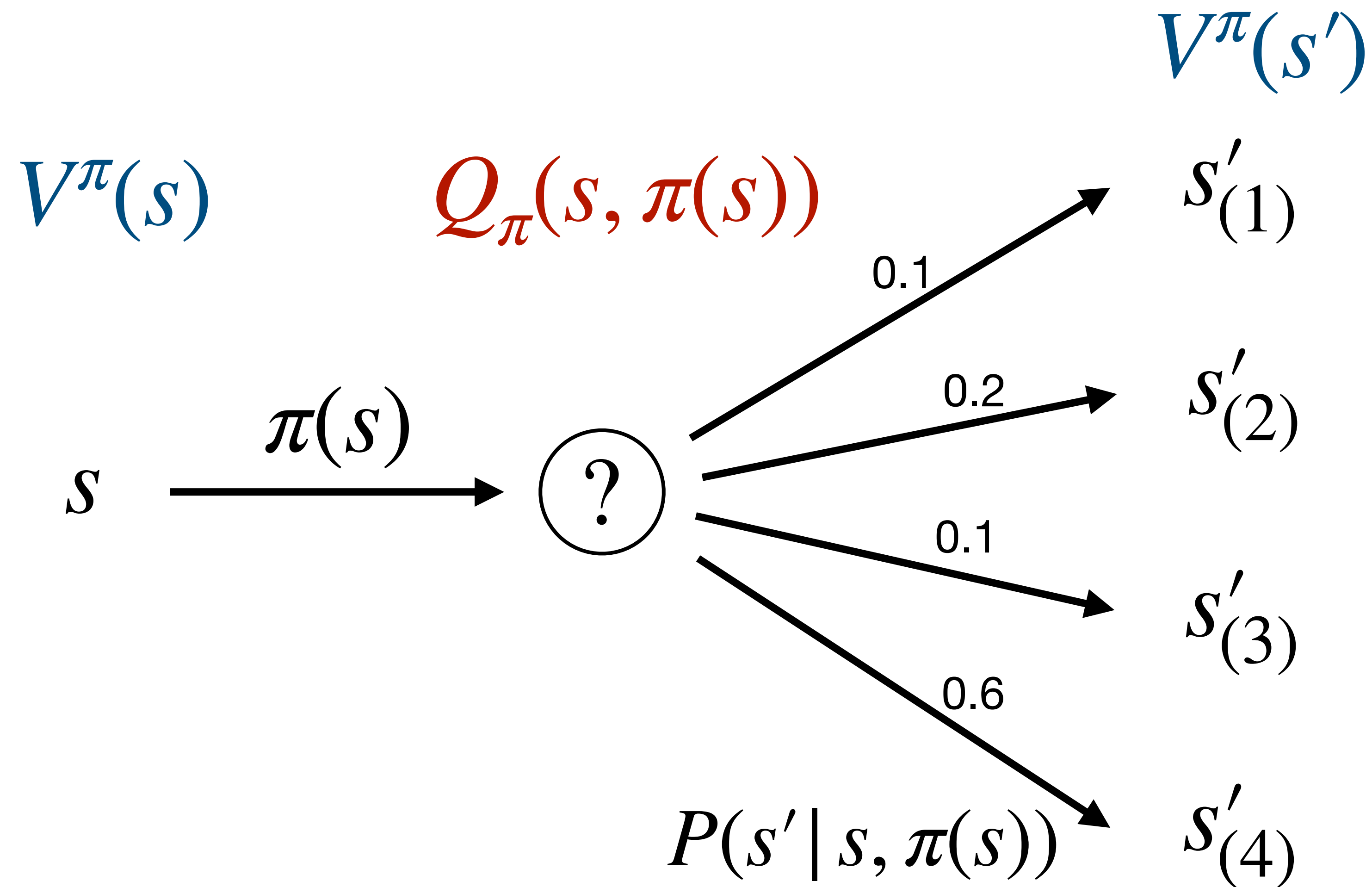
We can define a **Value Function**
associated with a Policy & Markov Decision Process

$$V^\pi(s) = \mathbb{E} \left[R(s_0) + \gamma R(s_1) + \gamma^2 R(s_2) + \dots \mid s_0 = s, \pi \right]$$

$V^\pi(s)$ is the **expected sum of discounted rewards** upon starting in state s , and taking actions according to π

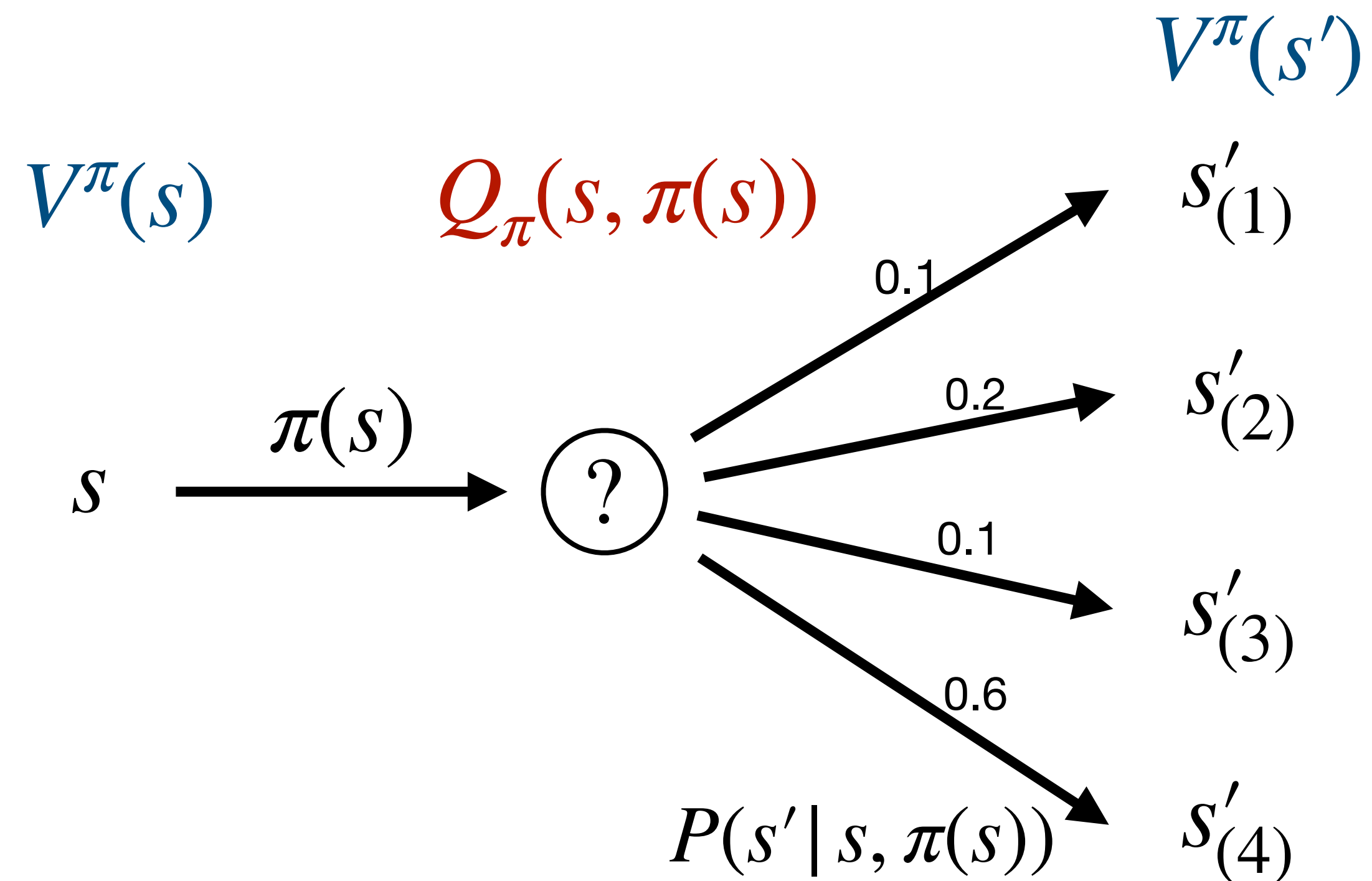
Q-Value of a Policy

Expected utility of taking action a from state s according to policy π



Q-Value of a Policy

Expected utility of taking action a from state s according to policy π



$$Q_\pi(s, a) = R(s) + \gamma \sum_{s'} P(s' | s, a) V^\pi(s')$$

How **good** is a policy π ?

We can define a **value function** associated with a policy & Markov Decision Process

$$V^\pi(s) = \mathbb{E} [R(s_0) + \gamma R(s_1) + \gamma^2 R(s_2) + \dots \mid s_0 = s, \pi]$$

$$V^\pi(s) = \mathbb{E} [R(s_0) + \gamma (R(s_1) + \gamma R(s_2) + \dots) \mid s_0 = s, \pi]$$

$$V^\pi(s) = R(s) + \gamma \mathbb{E} [(R(s_1) + \gamma R(s_2) + \dots) \mid s_0 = s, \pi]$$

$$V^\pi(s) = R(s) + \gamma \mathbb{E} [V^\pi(s_1) \mid s_0 = s, \pi]$$

$$V^\pi(s) = R(s) + \gamma \sum_{s' \in \mathcal{S}} P(s' \mid s, a) V^\pi(s') \quad \longrightarrow$$

s' is the state after s

The Bellman Equation

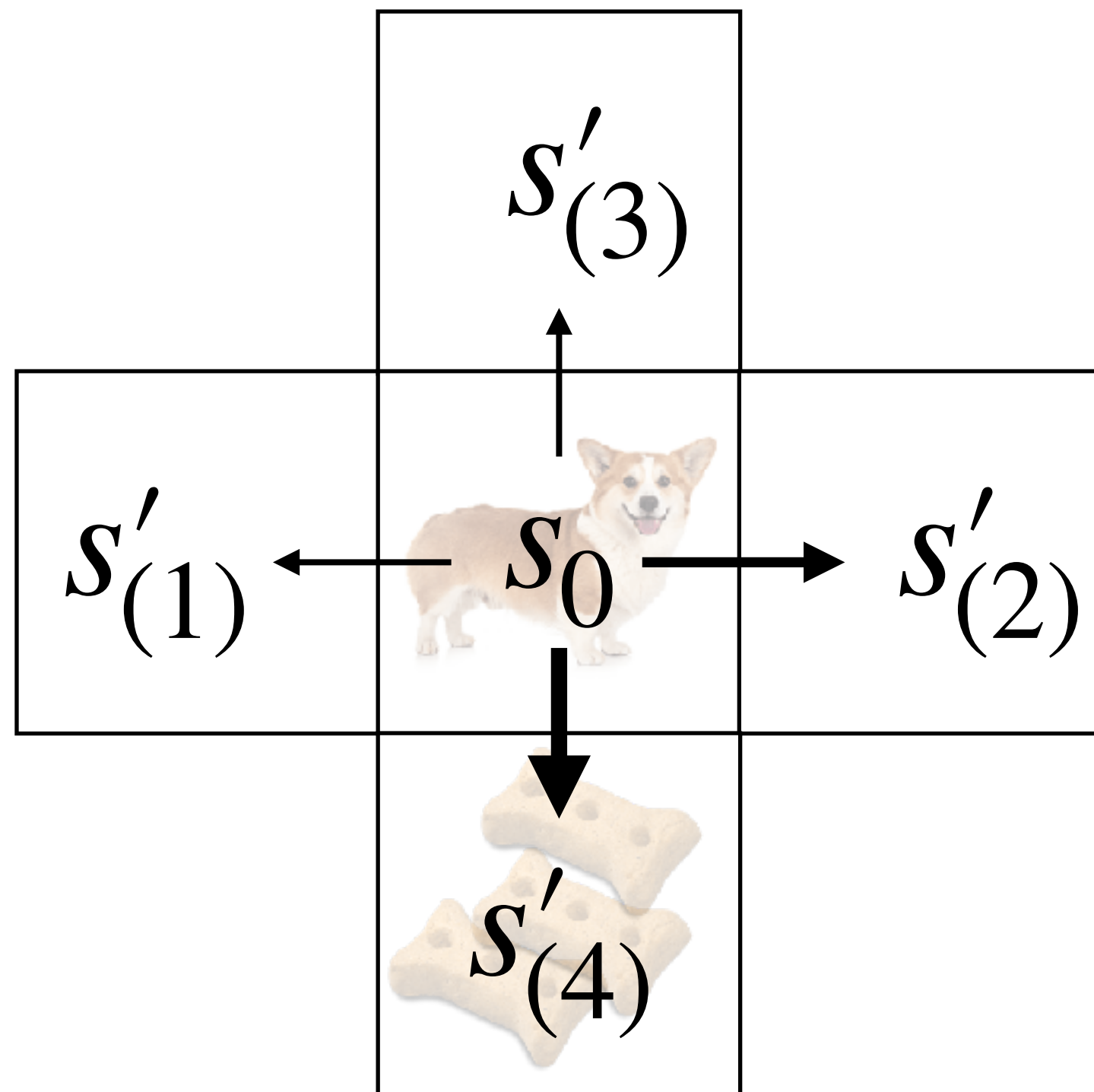
$$V^\pi(s) = R(s) + \gamma \sum_{s' \in \mathcal{S}} P(s' \mid s, \pi(s)) V^\pi(s')$$

Bellman Equation

$$V^\pi(s) = R(s) + \gamma \sum_{s' \in S} P(s' | s, \pi(s)) V^\pi(s')$$



Gives a system of $|S|$ linear equations for each unknown $V^\pi(s)$



Equation for $V^\pi(s_0)$

$$V^\pi(s_0) = R(s_0) + \gamma \left[P(s'_{(1)} | s_0, \pi(s_0)) V^\pi(s'_{(1)}) + P(s'_{(2)} | s_0, \pi(s_0)) V^\pi(s'_{(2)}) + P(s'_{(3)} | s_0, \pi(s_0)) V^\pi(s'_{(3)}) + P(s'_{(4)} | s_0, \pi(s_0)) V^\pi(s'_{(4)}) \right]$$

What's the **optimal value function**?

In other words, what's the **best expected sum of discounted rewards** if one gets to choose *any* policy π ?

$$V^{\star}(s) = \max_{\pi} V^{\pi}(s)$$

What's the **optimal policy** the agent can take if they start at state s ?

$$\pi^{\star}(s) = \arg \max_{a \in A} \sum_{s' \in S} P(s' | s, a) V^{\star}(s')$$

Solving for the optimal value function

Given the **optimal policy**:

$$\pi^{\star}(s) = \arg \max_{a \in A} \sum_{s' \in S} P(s' | s, a) V^{\star}(s')$$

We can write its associated **Bellman equation**:

$$V^{\star}(s) = R(s) + \gamma \max_{a \in A} \sum_{s' \in S} P(s' | s, a) V^{\star}(s')$$

How to **find** the optimal value function?

Given:

- Reward function $R(s)$
- State transition probability $P(s' | s, a)$

Value iteration algorithm

initialize $V(s) = 0$, for each state s

while not converged:

for every state s :

$$V(s) = R(s) + \gamma \max_{a \in A} \sum_{s'} P(s' | s, a) V(s')$$

end

end

Synchronous

compute new $V(s)$ for all s
then overwrite old $V(s)$

Asynchronous

update $V(s)$ one at a time

How to **find** the optimal **value function**?

Given:

- Reward function $R(s)$
- State transition probability $P(s' | s, a)$

$$\pi^*(s) = \arg \max_{a \in A} \sum_{s' \in S} P(s' | s, a) V^*(s')$$

Value iteration algorithm

initialize $V(s) = 0$, for each state s

while not converged:

for every state s :

$$V(s) = R(s) + \gamma \max_{a \in A} \sum_{s'} P(s' | s, a) V(s')$$

end

end

Having found V^* ,
compute optimal policy

Synchronous

compute new $V(s)$ for all s
then overwrite old $V(s)$

Asynchronous

update $V(s)$ one at a time

How to **find** the optimal **policy**?

Given:

- Reward function $R(s)$
- State transition probability $P(s' | s, a)$

$$\pi^\star(s) = \arg \max_{a \in A} \sum_{s' \in S} P(s' | s, a) V^\star(s')$$

Policy iteration algorithm

initialize π randomly

while not converged:

Let $V = V^\pi$ $\leftarrow$ (by solving linear system)

for every state s :

$$\pi(s) = \arg \max_{a \in A} \sum_{s'} P(s' | s, a) V(s')$$

end

end

Policy vs. Value Iteration

Policy iteration algorithm

```
initialize  $\pi$  randomly
while not converged:
    Let  $V = V^\pi$ 
    for every state  $s$ :
         $\pi(s) = \arg \max_{a \in A} \sum_{s'} P(s' | s, a) V(s')$ 
    end
end
```

Value iteration algorithm

```
initialize  $V(s) = 0$ , for each state  $s$ 
while not converged:
    for every state  $s$ :
         $V(s) = R(s) + \gamma \max_{a \in A} \sum_{s'} P(s' | s, a) V(s')$ 
    end
end
```

- There is no agreement which is better
- For small MDPs, policy iteration is very fast.
- But the linear solve step is slow for large state spaces, so value iteration is used more often

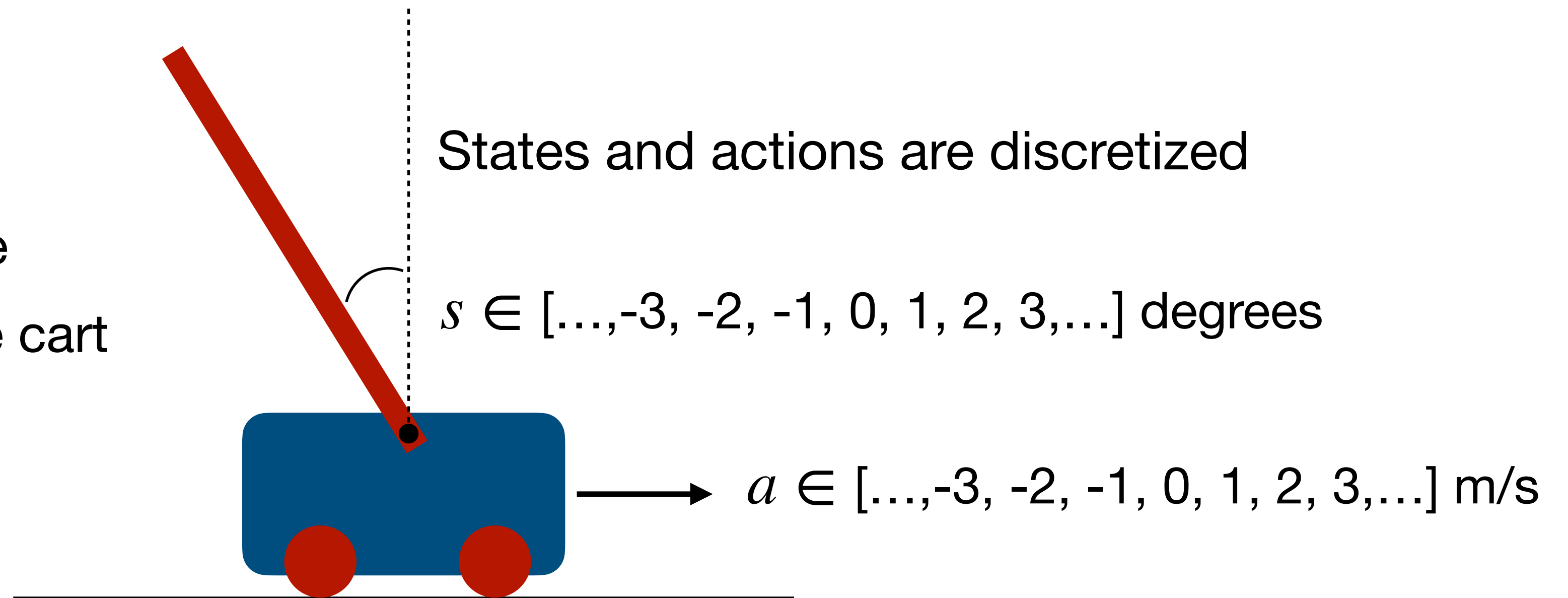
Examples

- Grid world problems: Frozen Lake Navigation, Cliff Walking, Taxi Problem
- Resource Management: Inventory (stocks) Control, Server Allocation, Battery Management (Electric Vehicle),
- Games: Blackjack, Chess, Tic-Tac-Toe, Pac-Man Navigation
- Finance & Trading: Portfolio Management, Option Pricing
- Robotics & Navigation: Drone Delivery, Robot Vacuum Cleaning, Robotic Arm Reaching
- Healthcare & Medicine: Treatment Planning, ICU Ventilator Management
- Learning & Education, Manufacturing, Environmental & Energy, Transportation

What if we don't know $P(s' | s, a)$ and $R(s)$?

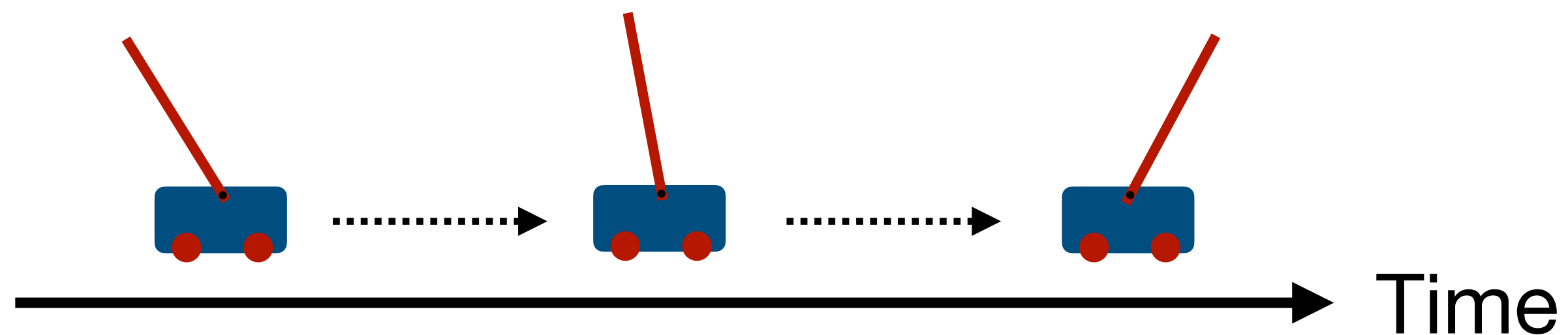
We are not given state transitions and rewards explicitly, but are given data

- State s is the angle of the pole
- Actions a is the velocity of the cart

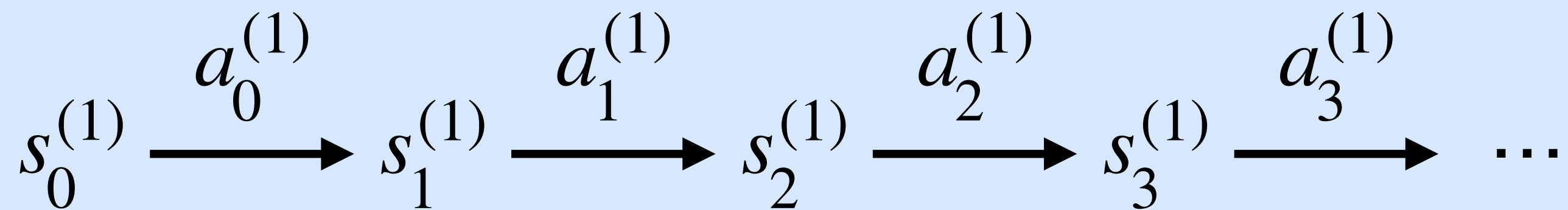


What if we don't know $P(s' | s, a)$ and $R(s)$?

We are given a number of trials



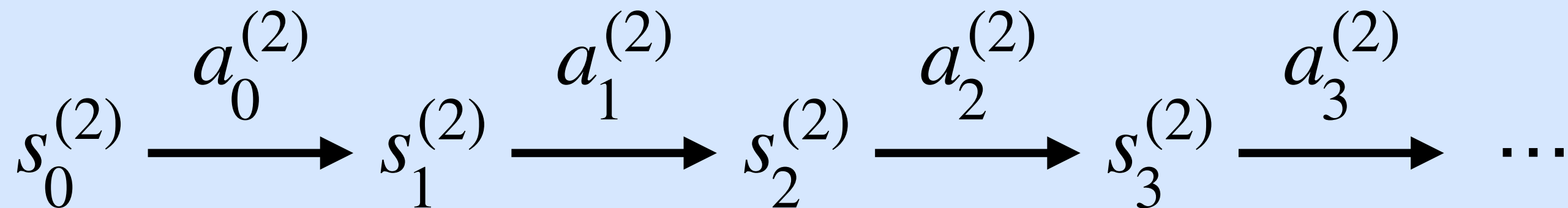
Trial 1



$s_i^{(j)}$: state at time i of trial j

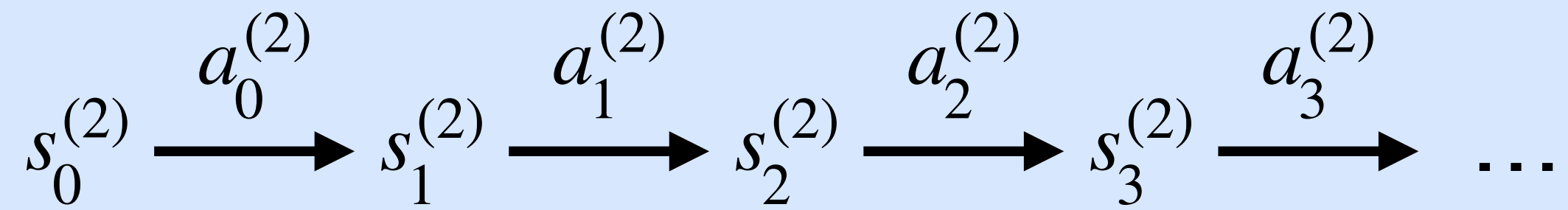
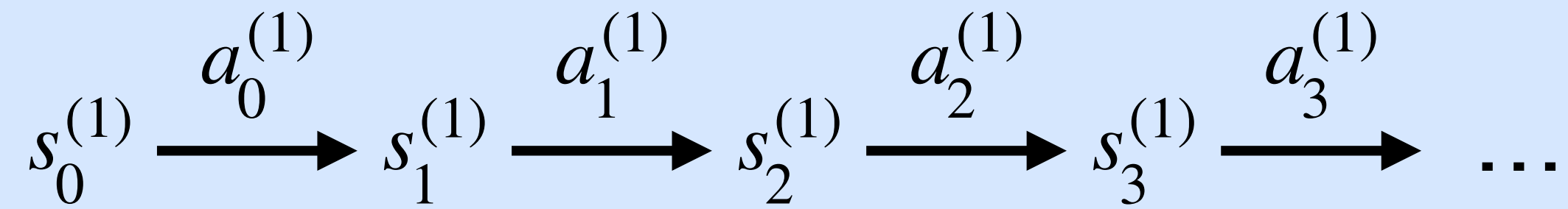
$a_i^{(j)}$: action taken from that state

Trial 2



$\vdots$ $\vdots$ $\vdots$ $\vdots$

Learning $P(s' | s, a)$



$\vdots$ $\vdots$ $\vdots$ $\vdots$

$$P(s' | s, a) = \frac{\# \text{ of } s \xrightarrow{a} s'}{\# \text{ of } s \xrightarrow{a}}$$

$$R(s) = \frac{\text{sum of } R \text{ in } s}{\# \text{ of times in } s}$$

Find the optimal policy with unknown P and R

initialize π randomly

for trials:

for trials:

 Execute π in MDP

end

 update estimates of $P(s' | s, a)$ and $R(s)$

 apply **value iteration** to find new $V(s)$

 update π

end

Model-Free Monte Carlo: on-policy

Expected utility of taking action a from state s according to policy π

$$Q_{\pi}(s, a) = R(s) + \gamma \sum_{s'} P(s' | s, a) V^{\pi}(s)$$

$$u(s_1) = R(s_1) + \gamma R(s_2) + \gamma^2 R(s_3)$$

$\hat{Q}_{\pi}(s_{i-1}, a_i)$ is average of $u(s_i)$ where $s_{i-1} = s$ and $a_i = a$ $= \mathbb{E}[u(s_i)]$

and (s, a) doesn't occur in $s_0, \dots, s_{i-2}$

Model-Free Monte Carlo: on-policy

Expected utility of taking action a from state s according to policy π

For each (s, a, u) :

$$\hat{Q}_{\pi}(s, a) \leftarrow (1 - \eta)\hat{Q}_{\pi}(s, a) + \eta u$$

For each (s, a, u) :

$$\hat{Q}_{\pi}(s, a) \leftarrow \hat{Q}_{\pi}(s, a) - \eta \left[\hat{Q}_{\pi}(s, a) - u \right]$$

SARSA: Using the Reward + Q-value

Reinforcement learning algorithms interpolate between new data (which specifies the target value) and the old estimate of the value (the prediction).

For each (s, a, R, s', a') :

$$\hat{Q}_{\pi}(s, a) \leftarrow (1 - \eta)\hat{Q}_{\pi}(s, a) + \eta[R(s) + \gamma\hat{Q}_{\pi}(s', a')]$$

Model-free Monte Carlo is **SARSA**, whose target is $R(s) + \gamma\hat{Q}_{\pi}(s', a')$.

SARSA's target is a combination of the data (the first step) and the estimate (for the rest of the steps).

In contrast, model-free Monte Carlo's u is taken purely from the data.

Q-Learning

Estimate $Q^\star$ with a model-free Monte Carlo approach

For each (s, a, R, s') :

$$\hat{Q}^\star(s, a) \leftarrow (1 - \eta)\hat{Q}^\star(s, a) + \eta[R + \gamma\hat{V}^\star(s')]$$

This is **rote learning**: doesn't generalize of (s, a)

Define features $\phi(s, a)$ s.t:
 $\hat{Q}^\star(s, a; \mathbf{w}) = \mathbf{w} \cdot \phi(s, a)$



All states that have similar features
will have similar Q-values

Q-Learning with function approximation

Estimate $Q^\star$ with a model-free Monte Carlo approach

For each (s, a, R, s') :

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \left[\hat{Q}^\star(s, a) - (R + \gamma \hat{V}^\star(s')) \right] \phi(s, a)$$

Implied objective function

Deep Reinforcement Learning

Policy Gradient: Train a policy $\pi(a | s)$

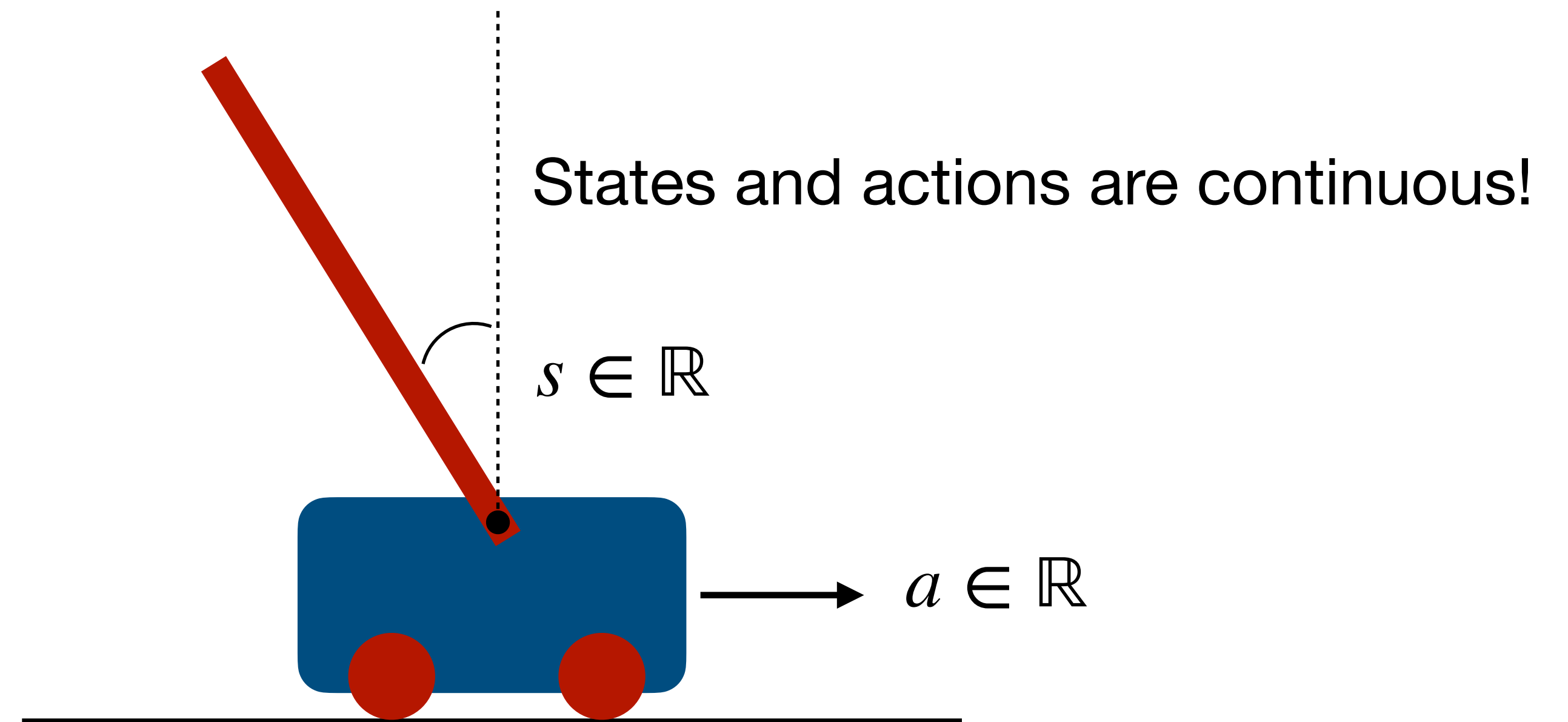
For each (s, a, R, s') :

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \left[\hat{Q}^{\star}(s, a) - (R + \gamma \hat{V}^{\star}(s')) \right] \phi(s, a)$$

Implied objective function

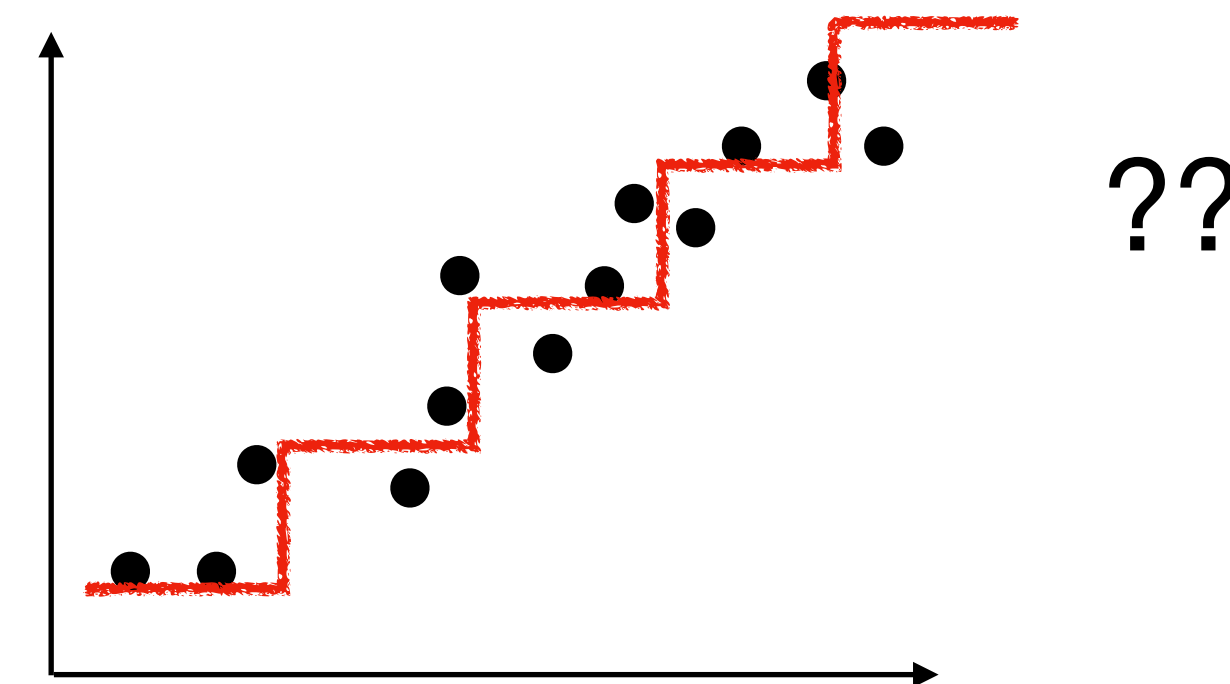
Continuous Markov Decision Processes (MDP)

- State s is the angle of the pole
- Actions a is the velocity of the cart



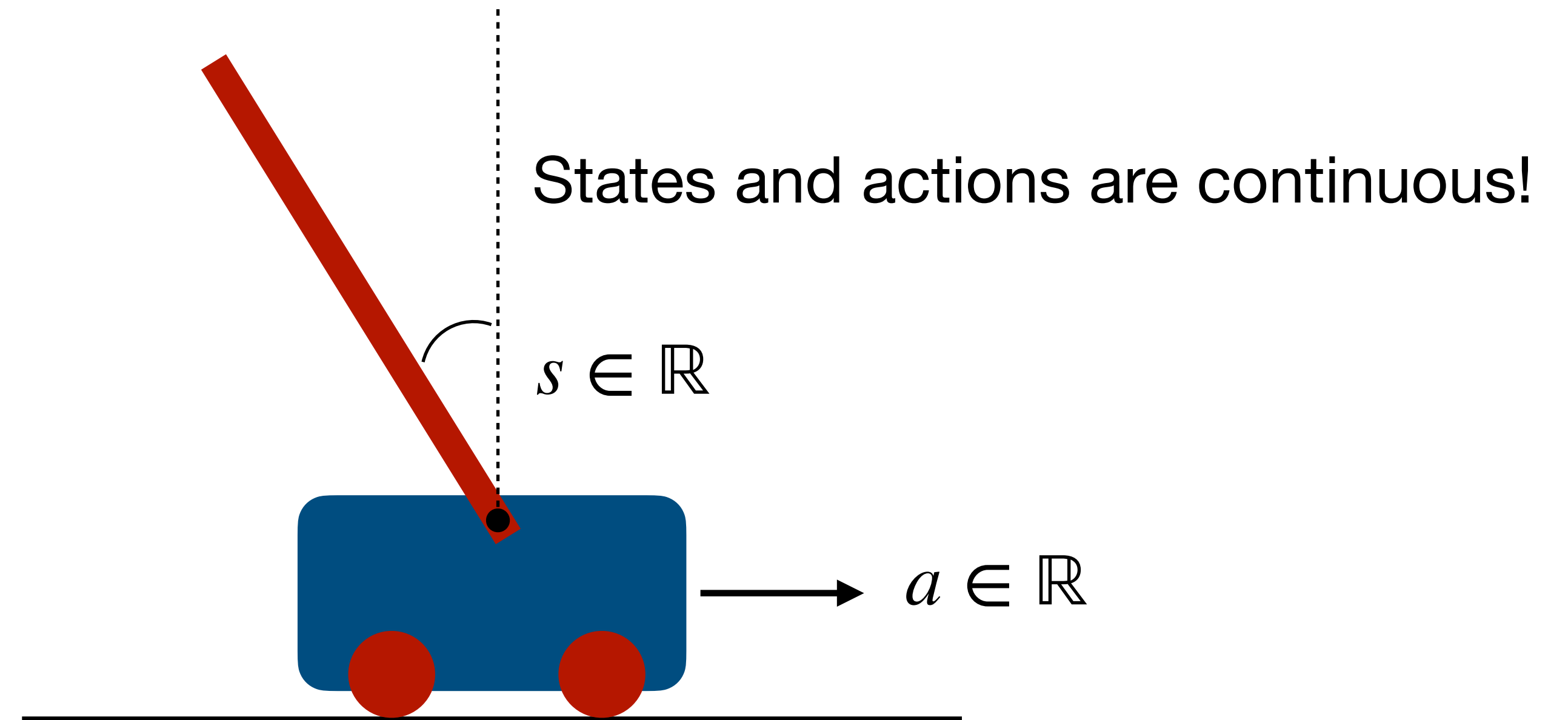
Downsides of discretization:

- Step-wise fit to a continuous problem
- Curse of dimensionality



Continuous Markov Decision Processes (MDP)

- State s is the angle of the pole
- Actions a is the velocity of the cart



Downsides of discretization:

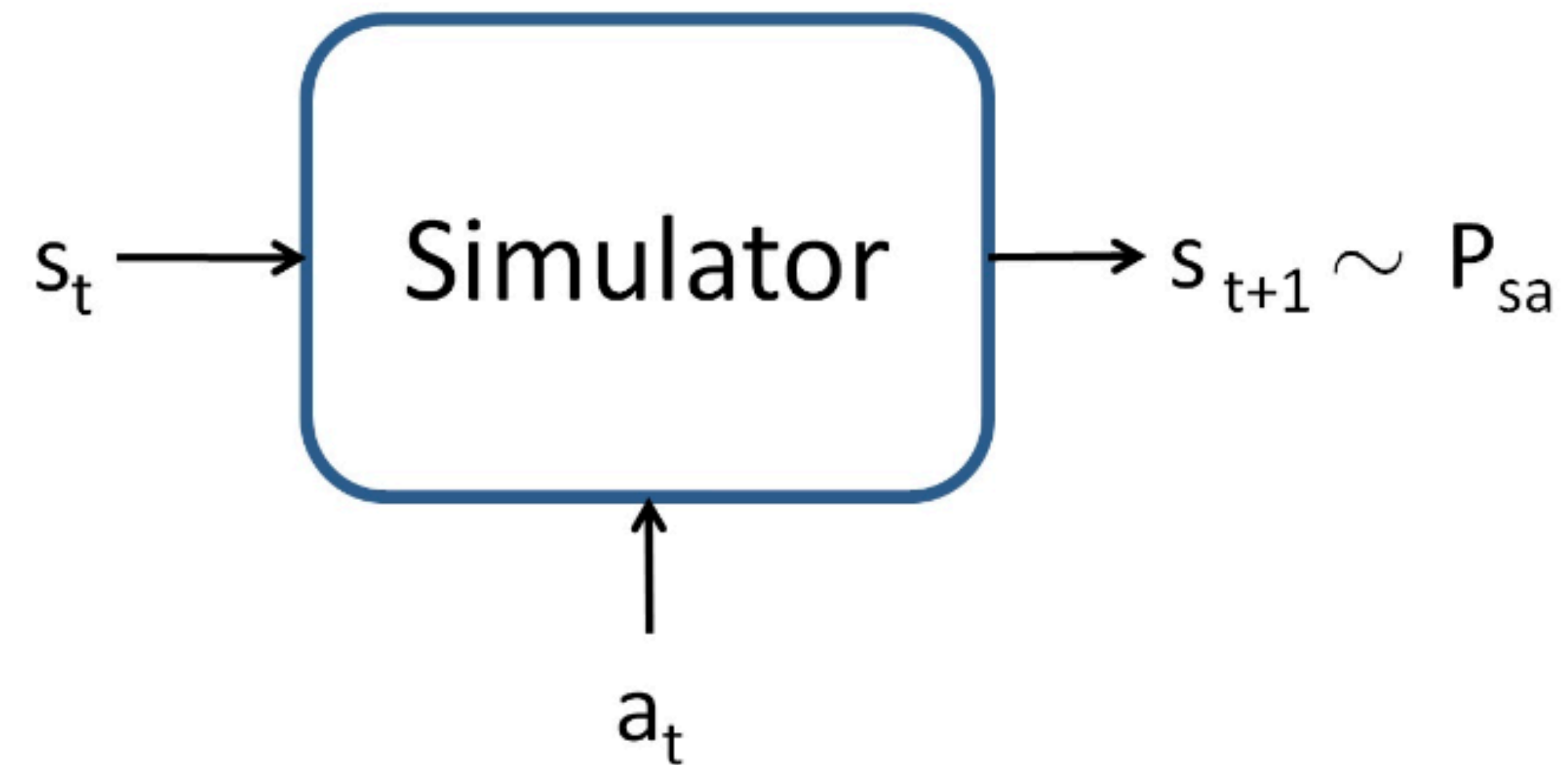
- Step-wise fit to a continuous problem
- Curse of dimensionality

Discretize each dim.
into k values

$$S \in \mathbb{R}^d \longrightarrow k^d \text{ States}$$

Too many to represent on a computer

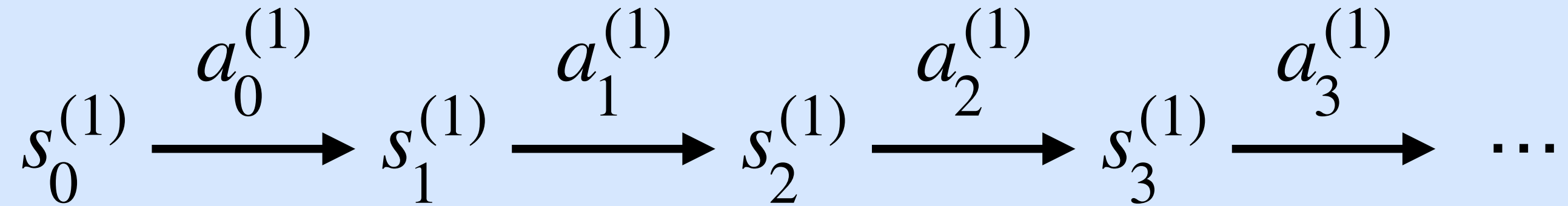
Value Function Approximation



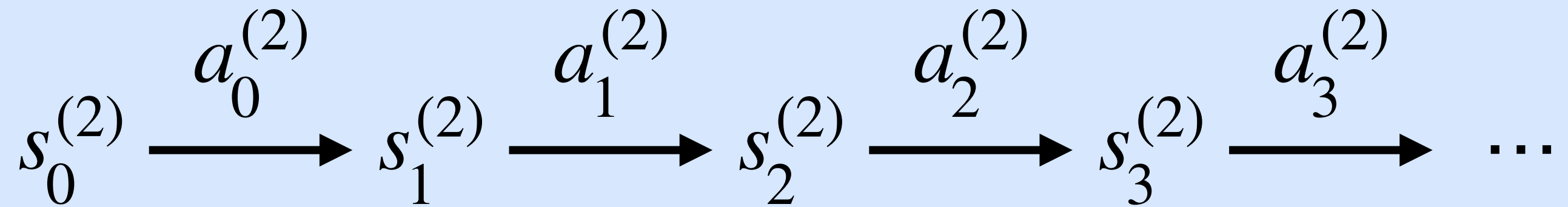
Given a physical model $\frac{ds}{dt} = f(s)$

Value Function Approximation

Trial 1



Trial 2



$\vdots$

$\vdots$

$\vdots$

$\vdots$

Learn a state-space model

$$s_{t+1} = As_t + Ba_t$$

With A and B as fitting parameters

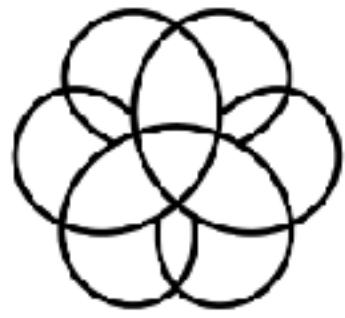
Fitting Value Iteration

$$\begin{aligned} V(s) &:= R(s) + \gamma \max_a \int_{s'} P_{sa}(s') V(s') ds' \\ &= R(s) + \gamma \max_a \mathbb{E}_{s' \sim P_{sa}} [V(s')] \end{aligned}$$

See page 188 for more

Open source models

python 3.8 | 3.9 | 3.10 | 3.11 | 3.12 pypi package 1.0.0 arXiv 2407.17032  pre-commit enabled code style black

 Gymnasium

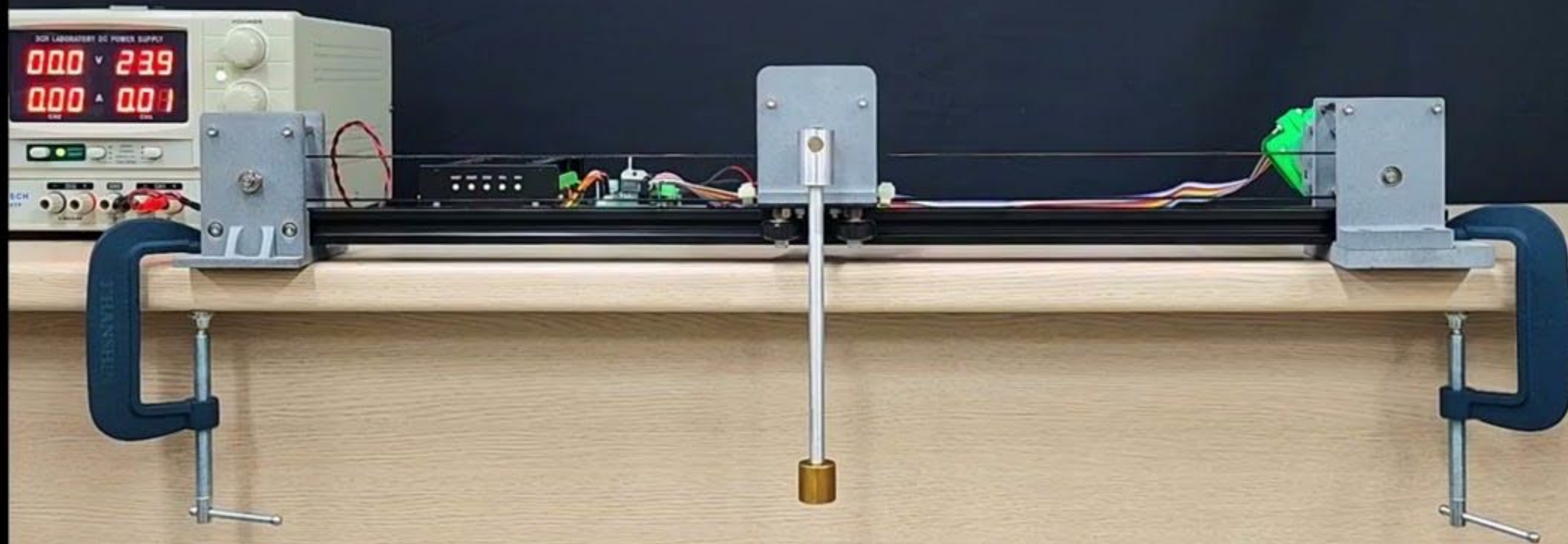
Gymnasium is an open source Python library for developing and comparing reinforcement learning algorithms by providing a standard API to communicate between learning algorithms and environments, as well as a standard set of environments compliant with that API. This is a fork of OpenAI's [Gym](#) library by its maintainers (OpenAI handed over maintenance a few years ago to an outside team), and is where future maintenance will occur going forward.

The documentation website is at gymnasium.farama.org, and we have a public discord server (which we also use to coordinate development work) that you can join here: <https://discord.gg/bnJ6kubTg6>

Environments

<https://github.com/Farama-Foundation/Gymnasium>

Since LW-RCP is based on Matlab/Simulink,
it is easy to observe real-time data from the
real system using the 'Scope' block.



<https://www.youtube.com/watch?v=IM6rYjM6HBU>

